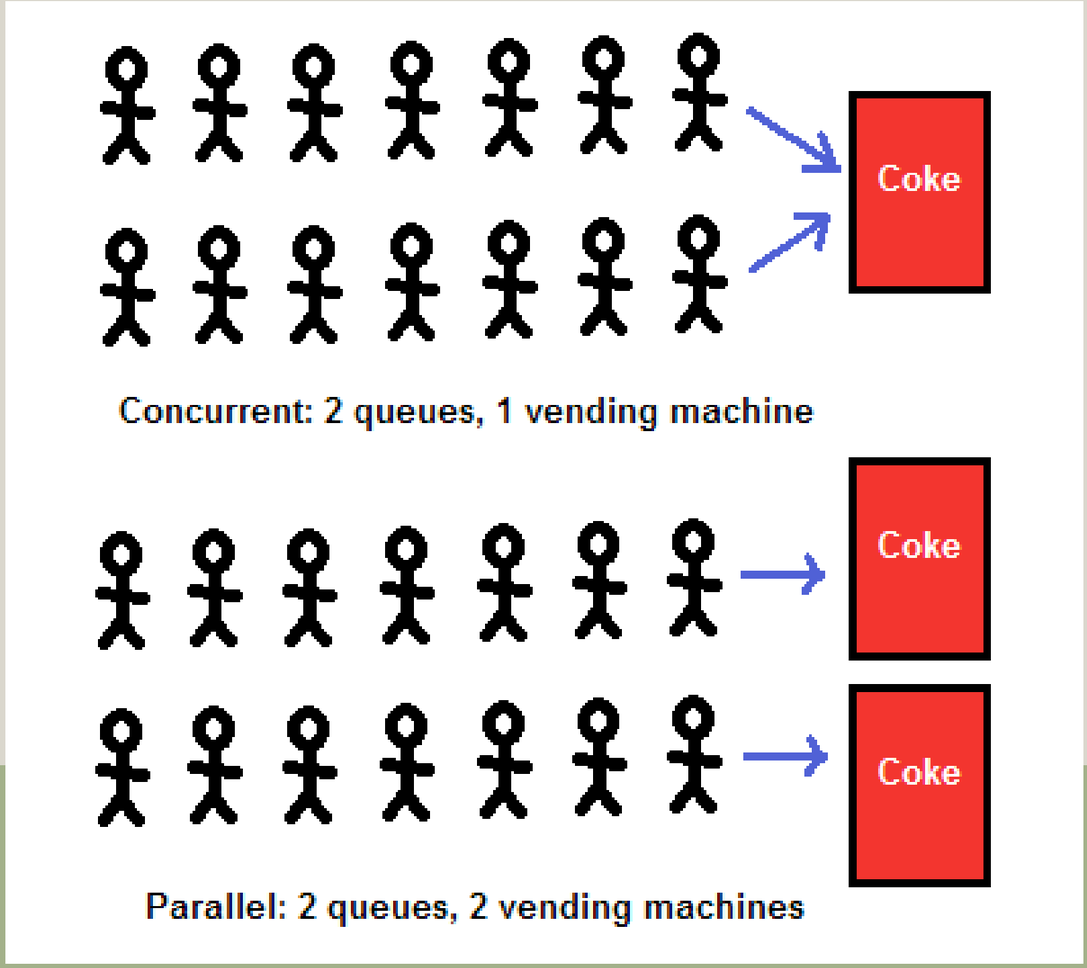
Coding Club, IIT Guwahati

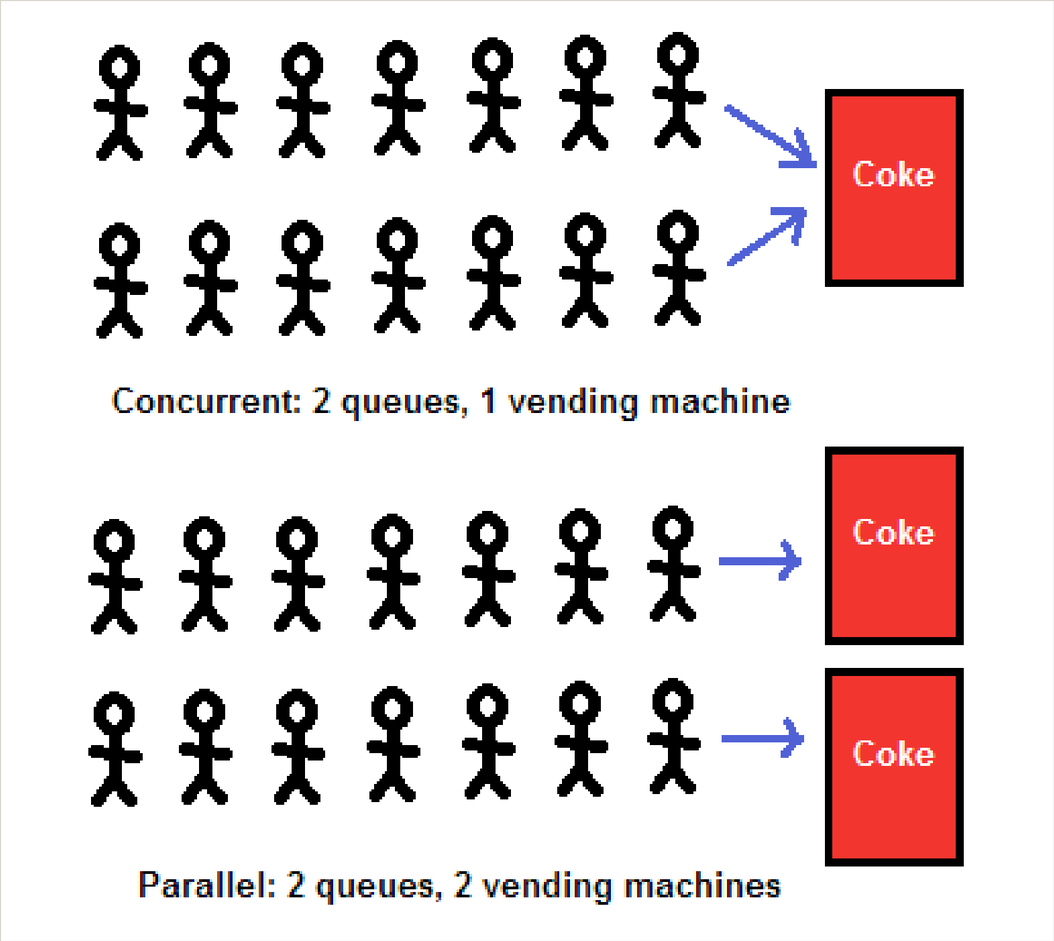
C++ Multithreading features

Session 1 : Concurrency, std::thread and more



Toshit Jain

What is Concurrency ?

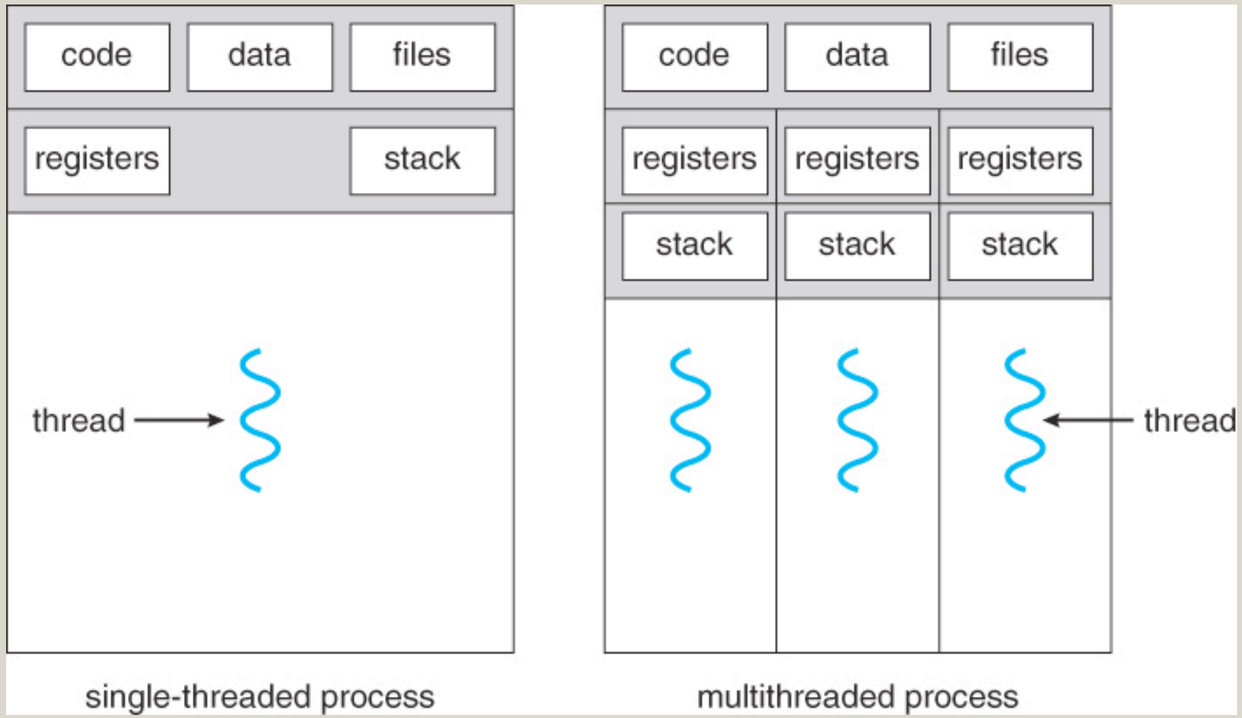


- At the simplest and most basic level, concurrency is about two or more separate activities happening at the same time.

- Historically, most computers have had one processor, with a single processing unit or core, and this remains true for many desktop machines today. Such a machine can really only perform one task at a time, but it can switch between tasks many times per second. By doing a bit of one task and then a bit of another and so on, it appears that the tasks are happening concurrently. This is called task switching.

- The important factor to consider is really the number of hardware threads: the measure of how many independent tasks the hardware can genuinely run concurrently.

What are threads ?



A thread is the smallest unit of execution within a process, representing a single sequence of instructions that can run independently. Multiple threads within the same process share the same memory space, resources, and code, but each thread has its own program counter, registers, and stack. This allows threads to run concurrently, improving performance and responsiveness, especially on multi-core systems, while requiring careful synchronization to avoid **data races** and inconsistencies.

Threads share memory

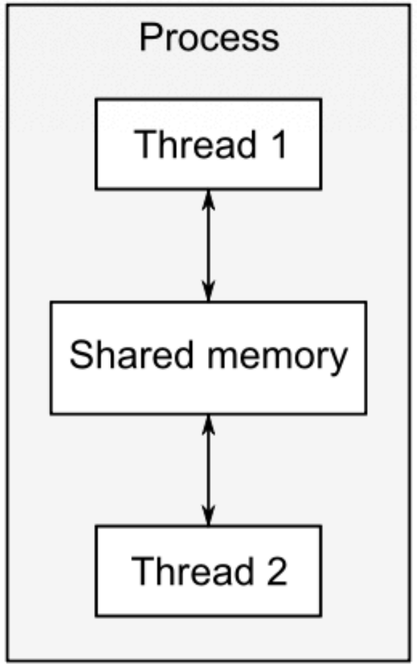


Figure 1.4 Communication between a pair of threads running concurrently in a single process

What is Concurrency ?

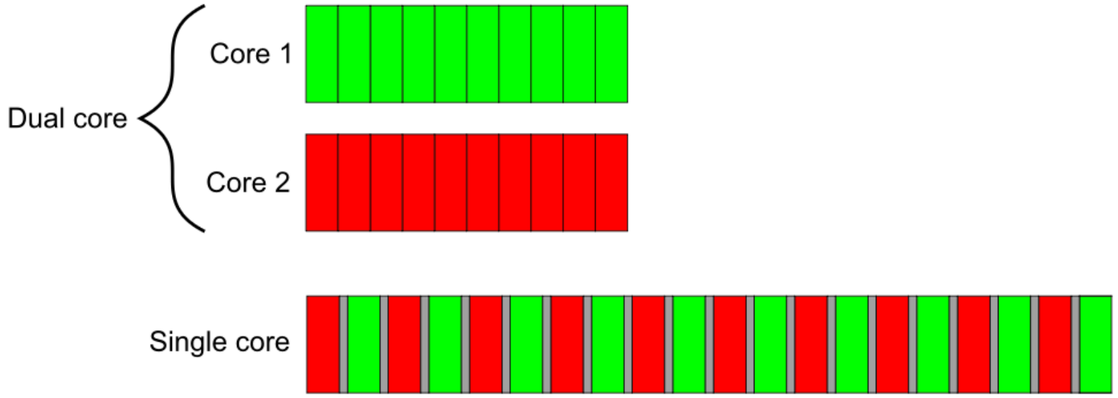


Figure 1.1 Two approaches to concurrency: parallel execution on a dual-core machine versus task switching on a single-core machine

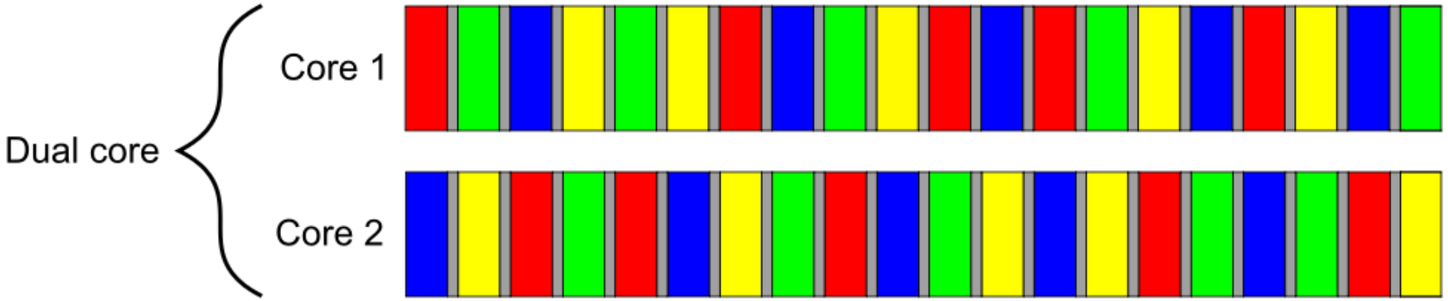


Figure 1.2 Task switching of four tasks on two cores

Why Concurrency ?

- The way to make use of concurrency within an application is to divide the application into multiple, separate, single threaded processes that are run at the same time, much as you can run your web browser and word processor at the same time. These separate processes can then pass messages to each other through all the normal inter-process communication channels. One downside is that such communication between processes is often either complicated to set up or slow or both, because operating systems typically provide a lot of protection between processes to avoid one process accidentally modifying data belonging to another process. Another downside is that there's an inherent overhead in running multiple processes: it takes time to start a process, the operating system must devote internal resources to manage the running processes.

- Thus, there are two main reasons to use concurrency in an application: separation of concerns and performance

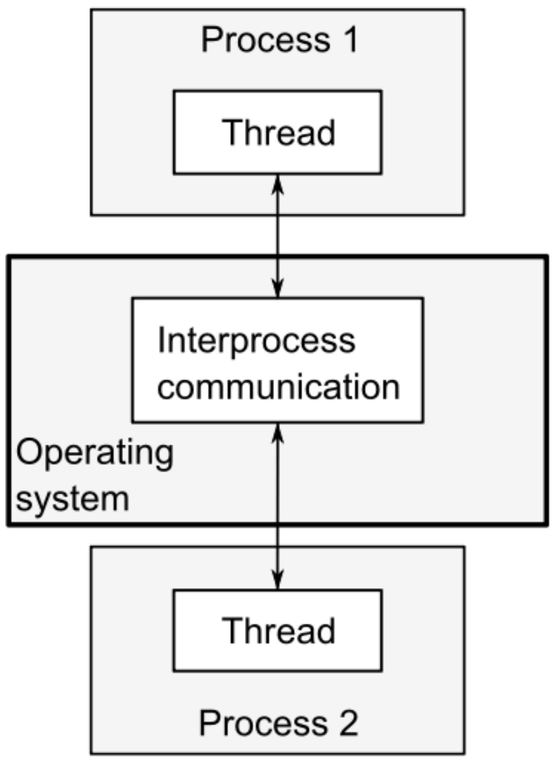


Figure 1.3 Communication between a pair of processes running concurrently

std::thread

- This library was included in the C++11 standard. So make sure you have a C++11-compatile compiler. It is included in the **<thread>** header.

- The **std::thread** is used to instantiate new threads of execution in a C++ program. This thread runs parallely (or concurrently depending on number of threads instantiated).

- The thread::join() function is called to wait for the thread to finish its execution.

- Try running a program without using thread::join()

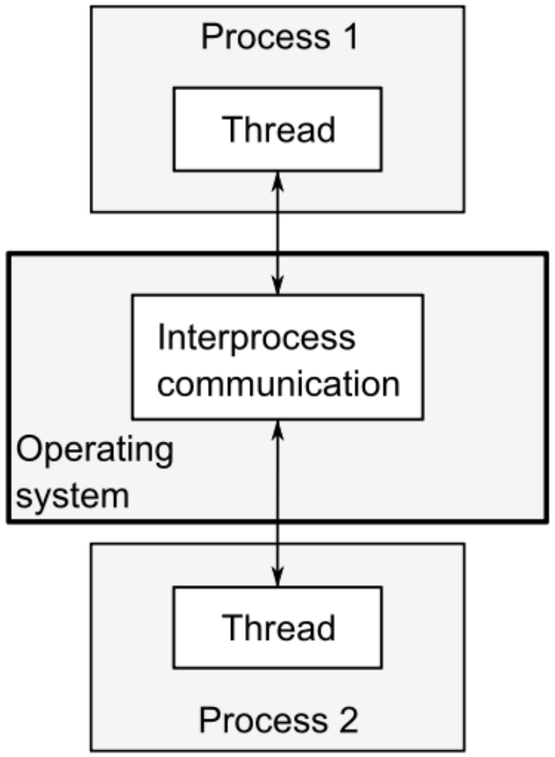


Figure 1.3 Communication between a pair of processes running concurrently

std::thread

main ×

```
1  #include <iostream>
2  #include <thread>
3
4  void hello(){
5    std::cout<<"Hello"<<'\n';
6  }
7
8  int main(){
9    std::thread t(hello);
10   t.join()
11 }
```

Invoking a C++ thread

- In C++, a thread can only be invoked using a **callable** type. The invokation can be done using the following methods

1. void-returning Function
2. Functor object
3. Lambda expressions

- The parameters can also be passed into the threads which we will see later. Note that thread cannot return anything. Any return can either be done using a **shared variable** or using **futures** which we will see later.

1. Pass parameters after all the callable related arguments are done.
2. References use **std::ref**
3. For member function of classes/structs, pass the member function pointer with the object reference.

Invoking a C++ thread

void-returning Function

main ×

```
1  #include <iostream>
2  #include <thread>
3
4  void hello(){
5    std::cout<<"Hello"<<'\n';
6  }
7
8  int main(){
9    std::thread t(hello);
10   t.join()
11 }
```

Invoking a C++ thread

Functor object

main

```cpp
#include <iostream>
#include <thread>

struct functor{
  void operator() (){
    std::cout<<"Hello"<<'\n';
  }
};

int main(){
  functor obj;
  std::thread t(obj);
  t.join();
}
```

Why didn't we use **std::thread t(functor())** ??

main

```cpp
#include <iostream>
#include <thread>

struct functor{
  void operator() (){
    std::cout<<"Hello"<<'\n';
  }
};

int main(){
  std::thread t{functor()};
  t.join();
}
```

Invoking a C++ thread

Lambda expression

main

```cpp
#include <iostream>
#include <thread>
#include <functional>

int main(){
  function<void(void)> lambda = [] () -> void {
    std::cout<<"Hello"<<'\n';
  };
  std::thread t(lambda);
  t.join();
}
```

main

```cpp
#include <iostream>
#include <thread>
#include <functional>

int main(){
  std::thread t( []{
    std::cout<<"Hello"<<'\n';
  });
  t.join();
}
```

Invoking a C++ thread

Adding parameters

```
#include <iostream>
#include <thread>

void function(int x, char y){
  // Something
}

int main(){
  int x = 5;
  char y = 'a';
  std::thread t(function,x,y);
  t.join();
}
```

What will happen if the function is **void member_func (unique_ptr<int>)** instead ??

```
#include <iostream>
#include <thread>

struct functor{
  char c;
  void member_func (int x){
    // Something
  }
};

int main(){
  functor obj;
  int x = 5;
  std::thread t(&functor::member_func,&obj,x);
  t.join();
}
```

Invoking a C++ thread

Adding parameters w/ reference

main

```cpp
#include <iostream>
#include <thread>

void function(int& y){
  // Something
}

int main(){
  int x = 5;
  std::thread t(function,x);
  // x doesn't change as function works
  // on internal copy of x
  t.join();
}
```

main

```cpp
#include <iostream>
#include <thread>

void function(int& y){
  // Something
}

int main(){
  int x = 5;
  std::thread t(function,std::ref(x));
  // Use std::ref instead
  // Now updates x
  t.join();
}
```

Thread completion & RAII

- We have two methods for thread completion

1. thread::join() → Waits for the thread to join
2. thread::detach() → Detaches the thread and moves forward
3. thread::joinable() is used to check whether we have joined the thread or not. We **cannot** call join()/detach() more than once on the same thread.
4. We *generally* use thread::join() only but thread::detach() also has its own benefits which we'll skip..

- Because of scope issues and calling join() more than once or missing it sometimes, we use RAII based **thread_guard** which calles join() whenever the std::thread object goes out of scope.

Thread completion & RAII

main ×

```cpp
#include <iostream>
#include <thread>

class thread_guard{
public:
    std::thread& _t;
    explicit thread_guard(std::thread& t) : _t(t) {} // explicit prevents implicit conversions
    // For example if a function is func(thread_guard tg) and we call func(std::thread t)
    // This implicitly guards the passed thread thus taking the ownership of the thread
    // And joining it at the end of function call

    ~thread_guard()
    {
        // Ensures join using RAII
        if(_t.joinable()) _t.join();
    }

    thread_guard(const thread_guard&) = delete; // Delete copy constructor
    thread_guard& operator=(const thread_guard&) = delete; // Delete assignment operator
    // These deletes ensure that not more than one guard is used for a thread
};
```

thread_guard class

Additional Points

- The std::thread cannot be copied. Thus, we use **std::move** to transfer ownership from one thread to another.

- Try to modify the **thread_guard** by using **std::thread** and **std::move** instead of **std::thread&** → This helps in actually taking the ownership of the thread.

- Other things to read → **choosing number of threads at runtime** and **thread ids**

All the things were taken from...

- C++ Concurrency in Action by Anthony Williams :

1. Chapter 1 : Hello, world of concurrency in C++!
2. Chapter 2 : Managing threads

Next session...

- Shared data, race conditions, using std::mutex and more

Thank You